** ACA – PROGRAMACIÓN AVANZADA

SISTEMA DE GESTIÓN DE BIBLIOTECA (BibliotecaApp)

PORTADA

Institución: CUN

Programa: Ingeniería de Sistemas Asignatura: Programación Avanzada Proyecto: Sistema de Gestión de Biblioteca – BibliotecaApp Presentado por: Johan Santiago Peña Garcia Docente: Veronica Castro Ciudad: Bogotá D. C. Fecha: 10 de noviembre de 2026

CONTRAPORTADA Sistema de Gestión de Biblioteca – BibliotecaApp

Proyecto desarrollado como evidencia del curso de Programación Avanzada, utilizando C#, Windows Forms y SQL Server. La aplicación permite administrar libros, autores, editoriales y usuarios; registrar y consultar préstamos; gestionar devoluciones desde el módulo de préstamos; y consultar reportes.

Estudiante: Johan Santiago Peña Garcia Programa: Ingeniería de Sistemas Institución: CUN Docente: Veronica Castro Año: 2026

TABLA DE CONTENIDO

1. Introducción

2. Objetivos

    - 2.1 Objetivo general
    - 2.2 Objetivos específicos

3. Planteamiento del problema

    - 3.1 Descripción del problema
    - 3.2 Justificación
    - 3.3 Alcance
    - 3.4 Fuera del alcance

4. Análisis de requerimientos

    - 4.1 Descripción general
    - 4.2 Actores del sistema
    - 4.3 Requisitos funcionales
    - 4.4 Requisitos no funcionales
    - 4.5 Reglas de negocio

5. Casos de uso

    - 5.1 Descripción
    - 5.2 Diagrama de casos de uso
    - 5.3 Descripción de casos de uso

6. Diagrama de clases

    - 6.1 Descripción
    - 6.2 Diagrama de clases
    - 6.3 Principales clases
    - 6.4 Aplicación de POO
7. Modelo entidad-relación
    - 7.1 Descripción
    - 7.2 Diagrama entidad-relación
    - 7.3 Relaciones principales
8. Diccionario de datos
9. Arquitectura del sistema
10. Explicación de los módulos desarrollados
11. Capturas de pantalla del sistema
12. Pruebas de funcionamiento
13. Conclusiones
14. Recomendaciones
15. Referencias bibliográficas
16. Anexos
17. INTRODUCCIÓN

El presente documento describe el desarrollo de BibliotecaApp, un Sistema de Gestión de Biblioteca diseñado como aplicación de escritorio para apoyar la administración de información bibliográfica y de usuarios. La solución fue desarrollada con C# y Windows Forms, conectada a una base de datos relacional en SQL Server.

La aplicación dispone de módulos para gestionar libros, autores, editoriales y usuarios, además de registrar y consultar préstamos, gestionar las devoluciones dentro del módulo de préstamos y generar reportes. El proyecto aplica conceptos de programación orientada a objetos, operaciones CRUD, validación de datos, manejo de excepciones y separación entre formularios y componentes de acceso a datos.

Este documento presenta el problema abordado, los requisitos, los casos de uso, los modelos de diseño, la estructura de datos, la arquitectura, la explicación de los módulos y las evidencias de funcionamiento. La documentación se debe mantener coherente con el código fuente y el script SQL entregados.

2. OBJETIVOS

2.1 Objetivo general Desarrollar una aplicación de escritorio para gestionar la información de una biblioteca mediante C#, Windows Forms y SQL Server, que permita administrar libros, autores, editoriales y usuarios, controlar préstamos y devoluciones y generar reportes.

2.2 Objetivos específicos

- Aplicar conceptos de programación orientada a objetos en la construcción de la aplicación.

- Diseñar e implementar una base de datos relacional en SQL Server.
- Implementar operaciones CRUD para las entidades administradas por el sistema.
- Establecer y probar la conexión entre Windows Forms y SQL Server.
- Validar los datos ingresados y manejar excepciones en las operaciones.
- Organizar los componentes de interfaz y acceso a datos de forma comprensible.
- Utilizar Git y GitHub para almacenar y versionar el proyecto.
- Documentar las características técnicas y funcionales de la solución.

3. PLANTEAMIENTO DEL PROBLEMA

3.1 Descripción del problema

La administración de una biblioteca requiere mantener organizada la información de sus libros, autores, editoriales, usuarios y préstamos. Cuando estos procesos se realizan de forma manual o con información dispersa, pueden presentarse dificultades para localizar registros, verificar datos y conocer el estado de los préstamos.

BibliotecaApp se desarrolló para centralizar esos registros en una base de datos y ofrecer una interfaz que permita realizar operaciones de gestión y consultas desde una aplicación de escritorio.

3.2 Justificación

El desarrollo de BibliotecaApp permite aplicar los conocimientos de Programación Avanzada en un escenario práctico. La aplicación integra una interfaz gráfica con una base de datos relacional, organiza los registros mediante entidades relacionadas y proporciona módulos para las operaciones de gestión. También constituye una práctica de estructuración de código, validación de información, acceso a datos y control de versiones.

3.3 Alcance

La versión desarrollada contempla:

- Gestión de libros.
- Gestión de autores.
- Gestión de editoriales.
- Gestión de usuarios.
- Registro y consulta de préstamos.
- Registro de devoluciones desde el módulo de préstamos.
- Módulo de reportes.
- Conexión con SQL Server.

3.4 Fuera del alcance

No se documentan funcionalidades que no hayan sido implementadas en la versión entregada. En particular, no se deben afirmar funcionalidades como gestión de categorías, multas automáticas, autenticación de usuarios o un módulo independiente de devoluciones a menos que estén presentes en el código.

4. ANÁLISIS DE REQUERIMIENTOS

4.1 Descripción general BibliotecaApp permite registrar, consultar, actualizar y eliminar información de las entidades principales de la biblioteca. Asimismo, facilita el registro y seguimiento de préstamos y las consultas disponibles mediante reportes.

4.2 Actores del sistema

Actor Descripción Bibliotecario / Administrador Persona que utiliza la aplicación para gestionar registros, realizar préstamos y devoluciones y consultar reportes.

4.3 Requisitos funcionales

Código Requerimiento Descripción RF01 Gestionar libros Registrar, consultar, actualizar y eliminar libros. RF02 Gestionar autores Registrar, consultar, actualizar y eliminar autores. RF03 Gestionar editoriales Registrar, consultar, actualizar y eliminar editoriales. RF04 Gestionar usuarios Registrar, consultar, actualizar y eliminar usuarios de la biblioteca. RF05 Gestionar préstamos Registrar y consultar préstamos asociados a un libro y un usuario. RF06 Gestionar devoluciones Registrar la devolución y actualizar los datos del préstamo conforme a la lógica implementada. RF07 Generar reportes Ejecutar los reportes disponibles en la aplicación. RF08 Navegar por el sistema Acceder desde el formulario principal a los módulos de libros, autores, editoriales, usuarios, préstamos y reportes. RF09 Conectar con SQL Server Permitir que la aplicación consulte y modifique la información almacenada en la base de datos appbiblioteca.

4.4 Requisitos no funcionales

Código Requerimiento Descripción RNF01 Usabilidad La interfaz debe presentar controles y opciones organizados para facilitar su uso. RNF02 Validación Los datos ingresados deben validarse de acuerdo con las reglas de cada operación. RNF03 Mantenibilidad El código debe estar organizado en formularios y clases con nombres descriptivos. RNF04 Acceso a datos La aplicación debe utilizar una conexión configurada con SQL Server. RNF05 Manejo de excepciones Los errores de ejecución y de acceso a datos deben tratarse de manera controlada. RNF06 Integridad Las relaciones y restricciones de la base de datos deben preservar la consistencia de los registros.

4.5 Reglas de negocio

Las reglas se deben contrastar con la implementación final y el script SQL:

- RN01: El código del libro debe ser único, según la restricción definida en la base de datos.
- RN02: Los campos definidos como obligatorios deben contener valores antes de guardar.
- RN03: Un préstamo debe referenciar un usuario existente.
- RN04: Un préstamo debe referenciar un libro existente.
- RN05: El estado del préstamo debe corresponder a la información de devolución registrada.
- RN06: Las cantidades disponibles de libros deben respetar las restricciones de integridad del esquema.
- RN07: Las operaciones no deben dejar datos inconsistentes.

5. CASOS DE USO

5.1 Descripción

Los casos de uso describen las acciones que realiza el actor Bibliotecario/Administrador al interactuar con los módulos de BibliotecaApp.

5.2 Diagrama de casos de uso

Insertar aquí el diagrama de casos de uso correspondiente a los módulos reales.

5.3 Descripción de casos de uso

CU01 – Gestionar libros

- Actor principal: Bibliotecario / Administrador.
- Descripción: Permite administrar los registros de libros.
- Precondición: La aplicación está abierta y la conexión con la base de datos está disponible.
- Flujo principal: El actor abre el módulo de libros, consulta los registros y selecciona una operación (registrar, modificar, eliminar o consultar). El sistema procesa la acción y presenta el resultado.
- Flujos alternativos: Los datos ingresados pueden no cumplir las validaciones o la operación puede generar un error de base de datos.
- Postcondición: La operación queda reflejada en la base de datos si se completa correctamente. CU02 – Gestionar autores y editoriales
- Actor principal: Bibliotecario / Administrador.
- Descripción: Permite administrar la información de autores y editoriales.
- Precondición: La aplicación está abierta.
- Flujo principal: El actor ingresa al módulo correspondiente, consulta los registros y ejecuta una operación CRUD.
- Flujos alternativos: Datos obligatorios faltantes, duplicidad o error de persistencia.
- Postcondición: Los cambios se reflejan en la base de datos si la operación es exitosa. CU03 – Gestionar usuarios
- Actor principal: Bibliotecario / Administrador.
- Descripción: Permite registrar y administrar usuarios de la biblioteca.
- Precondición: La aplicación está abierta.
- Flujo principal: El actor abre el módulo de usuarios, ingresa o selecciona datos y ejecuta una operación de gestión.
- Flujos alternativos: Datos inválidos o documento duplicado cuando la restricción aplique.
- Postcondición: El registro de usuario queda almacenado o actualizado. CU04 – Registrar préstamo
- Actor principal: Bibliotecario / Administrador.
- Descripción: Permite registrar un préstamo asociado a un usuario y un libro.
- Precondiciones: El usuario y el libro deben existir.
- Flujo principal: El actor abre el módulo de préstamos, selecciona usuario y libro, introduce las fechas requeridas y guarda la operación.
- Flujos alternativos: Selección incompleta, fechas inválidas o error al guardar.
- Postcondición: El préstamo queda registrado si la operación se completa. CU05 – Registrar devolución
- Actor principal: Bibliotecario / Administrador.
- Descripción: Permite registrar la devolución de un préstamo desde el módulo de préstamos.
- Precondición: Debe existir un préstamo susceptible de ser devuelto.
- Flujo principal: El actor selecciona el préstamo, registra la fecha de devolución y confirma la operación.
- Flujos alternativos: Préstamo no seleccionado o error al actualizar.
- Postcondición: El estado y la fecha de devolución se actualizan de acuerdo con la lógica implementada. CU06 – Consultar reportes
- Actor principal: Bibliotecario / Administrador.
- Descripción: Permite seleccionar y ejecutar los tipos de reportes disponibles.
- Precondición: La aplicación tiene acceso a la base de datos.
- Flujo principal: El actor abre reportes, selecciona el tipo y ejecuta la consulta.
- Postcondición: El sistema presenta el resultado del reporte.

6. DIAGRAMA DE CLASES

6.1 Descripción

El diagrama debe representar las clases que existen en el código fuente de BibliotecaApp, incluyendo los formularios y las clases de acceso a datos. Debe evitar incluir clases ficticias o que no se hayan implementado.

6.2 Diagrama de clases

DIAGRAMA EN CARPETA IMAGENES

6.3 Principales clases

Clase / componente Responsabilidad

Form1 Formulario principal y navegación hacia los módulos. Conexion Crear y probar la conexión con SQL Server. Formularios de gestión Presentar la interfaz para administrar libros, autores, editoriales, usuarios, préstamos y reportes. Clases de datos (...Datos) Ejecutar las operaciones de consulta y persistencia para los módulos correspondientes.

Completar los nombres exactos de las clases/formularios a partir del proyecto final antes de entregar.

6.4 Aplicación de POO

En el proyecto se utilizan clases para organizar responsabilidades, métodos para ejecutar operaciones y objetos para trabajar con componentes de la aplicación. El encapsulamiento se aplica mediante miembros definidos dentro de las clases. Documentar herencia, polimorfismo, colecciones o sobrecarga únicamente cuando se identifiquen en el código fuente.

7. MODELO ENTIDAD-RELACIÓN

7.1 Descripción

La base de datos appbiblioteca organiza los datos en tablas relacionadas mediante claves primarias y foráneas. El modelo debe reflejar el esquema real del script SQL entregado.

7.2 Diagrama entidad-relación Insertar aquí el diagrama ER correspondiente a las tablas reales.

7.3 Relaciones principales

- Libros se relaciona con Editoriales mediante id_editorial.
- Libros y Autores se relacionan mediante la tabla intermedia LibroAutor.
- Prestamos se relaciona con Libros mediante id_libro.
- Prestamos se relaciona con Usuarios mediante id_usuarios. Las cardinalidades exactas deben representarse conforme a las claves foráneas del script SQL.

8. DICCIONARIO DE DATOS

8.1 Convenciones

Abreviatura Significado PK Clave primaria FK Clave foránea UQ Restricción de unicidad NOT NULL Campo obligatorio NULL Campo que puede aceptar valor nulo

8.2 Tabla Editoriales

Campo Tipo Clave / restricción Nulo Descripción id_editorial INT IDENTITY PK No Identificador de editorial. nombre VARCHAR(100) — No Nombre de la editorial. pais VARCHAR(60) — Sí País asociado a la editorial.

8.3 Tabla Autores

Campo Tipo Clave / restricción Nulo Descripción id_autor INT IDENTITY PK No Identificador del autor. nombre VARCHAR(80) — No Nombre del autor. apellido VARCHAR(80) — No Apellido del autor. nacionalidad VARCHAR(60) — Sí Nacionalidad del autor.

8.4 Tabla Libros

Campo Tipo Clave / restricción Nulo Descripción id_libro INT IDENTITY PK No Identificador del libro. codigo_libro VARCHAR(20) UQ No Código único del libro. titulo VARCHAR(150) — No Título del libro. ISBN VARCHAR(20) — Sí ISBN del libro. fecha_publicacion DATE — Sí Fecha de publicación. categoria VARCHAR(80) — Sí Categoría textual del libro. id_editorial INT FK No Editorial relacionada. cantidad INT CHECK >= 0 No Cantidad total de ejemplares. cantidad_disponible INT CHECK entre 0 y cantidad No Ejemplares disponibles.

8.5 Tabla LibroAutor

Campo Tipo Clave / restricción Nulo Descripción id_libro INT FK No Libro relacionado. id_autor INT FK No Autor relacionado.

Verificar si el script define una clave primaria compuesta u otra restricción para esta tabla y documentarla aquí.

8.6 Tabla Usuarios

Campo Tipo Clave / restricción Nulo Descripción id_usuarios INT IDENTITY PK No Identificador del usuario. NOMBRES [Tipo exacto del script] — [Verificar] Nombres del usuario. APELLIDOS [Tipo exacto del script] — [Verificar] Apellidos del usuario. DOCUMENTO [Tipo exacto del script] UQ No Documento único del usuario. TELEFONO [Tipo exacto del script] — [Verificar] Teléfono de contacto. CORREO [Tipo exacto del script] — [Verificar] Correo electrónico.

8.7 Tabla Prestamos

Campo Tipo Clave / restricción Nulo Descripción id_prestamo INT IDENTITY PK No Identificador del préstamo. FECHA_DE_PRESTAMO [Tipo exacto del script] — [Verificar] Fecha de registro del préstamo. FECHA_ESTIMADA_DEVOL [Tipo exacto del script] — [Verificar] Fecha estimada de devolución. FECHA_DEVOLUCION [Tipo exacto del script] — [Verificar] Fecha efectiva de devolución, cuando exista. ESTADO [Tipo exacto del script] — [Verificar] Estado del préstamo. id_libro INT FK No Libro prestado. id_usuarios INT FK No Usuario que realiza el préstamo.

Nota: Los tipos de datos y valores NULL marcados como “Verificar” deben completarse consultando el script SQL definitivo. No entregar el diccionario con estos marcadores.

9. ARQUITECTURA DEL SISTEMA

9.1 Descripción general

BibliotecaApp organiza la aplicación mediante formularios Windows Forms y clases de acceso a datos. La clase Conexion proporciona el mecanismo de conexión con SQL Server y los componentes de datos realizan

operaciones sobre las tablas. La documentación de capas debe describir la organización real del código, sin presentar una capa de negocio independiente si no existe físicamente.

9.2 Diagrama de arquitectura

DIAGRAMA EN CARPETA IMAGENES

9.3 Presentación

La capa de presentación está compuesta por el formulario principal y los formularios de los módulos. Permite al usuario introducir datos, seleccionar operaciones y visualizar los resultados.

9.4 Acceso a datos

Las clases del espacio de nombres BibliotecaApp.Datos encapsulan las operaciones de acceso a la base de datos. Se utiliza Microsoft.Data.SqlClient para la comunicación con SQL Server.

9.5 Base de datos

- Motor: Microsoft SQL Server.
- Base de datos: appbiblioteca.
- Tablas: Editoriales, Autores, Libros, LibroAutor, Usuarios y Prestamos.

9.6 Flujo de información

1. El usuario selecciona una acción en un formulario.
2. El formulario valida los datos que correspondan.
3. La clase de datos ejecuta la operación mediante la conexión configurada.
4. SQL Server procesa la consulta y devuelve el resultado.
5. La interfaz presenta la información o el mensaje correspondiente.
6. EXPLICACIÓN DE LOS MÓDULOS DESARROLLADOS

10.1 Módulo principal

Permite navegar a los módulos de libros, autores, editoriales, usuarios, préstamos y reportes.

10.2 Módulo de libros

Permite administrar la información de los libros registrados, incluyendo su código, título, datos bibliográficos, editorial, cantidades y disponibilidad, conforme a los campos del sistema.

10.3 Módulo de autores

Permite registrar, consultar, actualizar y eliminar autores, almacenando sus nombres, apellidos y nacionalidad.

10.4 Módulo de editoriales

Permite administrar las editoriales relacionadas con los libros, incluyendo su nombre y país. 10.5 Módulo de usuarios

Permite administrar los usuarios de la biblioteca mediante sus datos personales y de contacto, de acuerdo con los campos implementados.

10.6 Módulo de préstamos y devoluciones

Permite registrar y consultar préstamos asociados a usuarios y libros. La devolución se gestiona dentro del módulo de préstamos, actualizando los campos correspondientes del registro según la funcionalidad implementada.

10.7 Módulo de reportes

Permite seleccionar el tipo de reporte disponible y presentar los resultados de las consultas implementadas. Añadir aquí los nombres exactos de los reportes que aparecen en el formulario final.

10.8 Conexión con SQL Server

La aplicación utiliza la clase Conexion para crear conexiones con SQL Server a través de Microsoft.Data.SqlClient. La cadena de conexión configurada utiliza el servidor local, la base de datos appbiblioteca, autenticación integrada de Windows y la opción de confiar en el certificado del servidor para la conexión local.

La conexión fue probada durante la configuración del proyecto y se obtuvo el mensaje de conexión exitosa. Para ejecutar el sistema en otro equipo, se debe tener SQL Server instalado, restaurar o crear la base de datos mediante el script entregado y adaptar el servidor en la cadena de conexión según el entorno.

11. CAPTURAS DE PANTALLA DEL SISTEMA

Se dejan capturas de mi aplicacion en la carpeta imagenes

11.1 Menú principal

Descripción: Formulario principal desde el cual se accede a los módulos de la aplicación. 11.2 Gestión de libros

Descripción: Interfaz para consultar y administrar los registros de libros. 11.3 Gestión de autores

Descripción: Interfaz para administrar los autores. 11.4 Gestión de editoriales

Descripción: Interfaz para administrar las editoriales. 11.5 Gestión de usuarios

Descripción: Interfaz para administrar los usuarios de la biblioteca. 11.6 Gestión de préstamos y devoluciones

Descripción: Interfaz para registrar y consultar préstamos y gestionar devoluciones. 11.7 Reportes

Descripción: Interfaz para seleccionar y visualizar los reportes implementados.

12. PRUEBAS DE FUNCIONAMIENTO

12.1 Estrategia de pruebas Las pruebas deben realizarse sobre la aplicación ejecutable y la base de datos configurada. Para cada caso se debe registrar la acción, el resultado esperado, el resultado observado y el estado final. Completar los resultados con evidencias reales; no marcar como aprobado un caso que no se haya ejecutado.

12.2 Casos de prueba

ID Funcionalidad Entrada / acción Resultado esperado Resultado obtenido Estado CP01 Conexión SQL Server Iniciar la aplicación con la conexión configurada. La conexión se establece correctamente. [Completar con resultado observado] [Pendiente] CP02 Registrar libro Ingresar datos válidos y guardar. El libro queda registrado y aparece en el listado. [Completar] [Pendiente] CP03 Validación de libro Ingresar datos incompletos o un código duplicado, si aplica. La aplicación impide la operación y muestra un mensaje. [Completar] [Pendiente] CP04 Registrar autor Ingresar datos válidos. El autor queda registrado. [Completar] [Pendiente] CP05 Registrar editorial Ingresar datos válidos. La editorial queda registrada. [Completar] [Pendiente] CP06 Registrar usuario Ingresar datos válidos. El usuario queda registrado. [Completar] [Pendiente] CP07 Registrar préstamo Seleccionar usuario y libro y completar los datos. El préstamo queda registrado. [Completar] [Pendiente] CP08 Registrar devolución Seleccionar un préstamo y registrar devolución. El registro del préstamo se actualiza correctamente. [Completar] [Pendiente] CP09 Ejecutar reporte Seleccionar un tipo de reporte. El sistema presenta el resultado correspondiente. [Completar] [Pendiente]

12.3 Evidencias de pruebas

se dejan eviddencias en la carpeta de imagen de todos los CRUDS correctamente ejecutados

Insertar capturas que demuestren los casos de prueba ejecutados, especialmente conexión, operación CRUD, préstamo, devolución y reporte. Las capturas deben corresponder a resultados reales.

13. CONCLUSIONES

14. El desarrollo de BibliotecaApp permitió integrar una aplicación Windows Forms con una base de datos SQL Server para administrar información de una biblioteca.

15. El proyecto permitió aplicar operaciones CRUD y conceptos de programación orientada a objetos en módulos funcionales.

16. La estructura de tablas y sus relaciones permiten organizar la información de libros, autores, editoriales, usuarios y préstamos.

17. La documentación y las pruebas finales deben mantenerse alineadas con la implementación entregada y las evidencias obtenidas.

18. RECOMENDACIONES

- Mantener copias de seguridad periódicas de la base de datos.
- Verificar y documentar las validaciones de los formularios antes de realizar operaciones de persistencia.
- Mantener el script SQL actualizado junto con el código fuente.
- Continuar utilizando commits descriptivos para registrar cambios en el repositorio.
- Como mejoras futuras, considerar nuevas consultas o funcionalidades únicamente después de definir y probar su implementación.

15. REFERENCIAS BIBLIOGRÁFICAS Incluir solamente fuentes que hayan sido consultadas durante el desarrollo. Verificar fecha, título y URL de cada referencia antes de entregar.

- Microsoft. (s. f.). Documentación de Windows Forms para .NET. Microsoft Learn. https://learn.microsoft.com/dotnet/desktop/winforms/
- Microsoft. (s. f.). Microsoft.Data.SqlClient namespace. Microsoft Learn. https://learn.microsoft.com/dotnet/api/microsoft.data.sqlclient
- Microsoft. (s. f.). SQL Server documentation. Microsoft Learn. https://learn.microsoft.com/sql/

16. ANEXOS Anexo A. Repositorio GitHub Repositorio: (https://github.com/santiaguito1080-eng/BibliotecaApp) Anexo B. Script/Scripts appbiblioteca.sql Archivo: appbiblioteca.sql

El script debe incluir la creación de la base de datos, tablas, claves, relaciones, restricciones y datos de prueba conforme al esquema final. Anexo C. Evidencia de Git y GitHub

Insertar capturas reales de:

- Repositorio creado.
- Commits realizados durante el desarrollo.
- Organización de los archivos.
- Código fuente y script SQL publicados.
- Archivo README.md.

Anexo D. Evidencias adicionales

[Agregar capturas o documentos complementarios pertinentes.] LISTA DE VERIFICACIÓN ANTES DE ENTREGAR

- [x] Portada y contraportada completadas con los datos del docente y grupo.
- [x] Introducción y objetivos revisados.
- [x] Tabla de contenido actualizada.
- [x] Numeración de páginas aplicada en la exportación final.
- [x] Planteamiento del problema, justificación y alcance revisados.
- [x] Requisitos funcionales y no funcionales alineados con la aplicación.
- [x] Casos de uso y diagrama incluidos.
- [x] Diagrama de clases validado contra el código fuente.
- [x] Modelo entidad-relación validado contra el script SQL.
- [x] Diccionario de datos completado con tipos exactos, restricciones y nulabilidad.
- [x] Arquitectura y módulos descritos fielmente.
- [x] Capturas reales agregadas.
- [x] Pruebas ejecutadas y resultados observados consignados.
- [x] Conclusiones y recomendaciones revisadas.
- [x] Referencias APA revisadas.
- [x] Código fuente completo incluido.
- [x] Script SQL y datos de prueba incluidos.
- [x] README.md con instrucciones de ejecución.
- [x] Capturas de GitHub y commits incluidas.